Off the Bench (Short Version)

Build an AI-assisted sports technique analyzer with Python, MediaPipe and Gradio

Student tutorial: follow the steps in order and build the project as you read.

Contents

What you're building 3

Setup 4

The big picture 5

Build the features 6

Feature 1: Sports and skill levels 6

Feature 2: From measurements to coaching 6

Feature 3: Personalize the AI prompt 7

Optional extensions 8

What you're building

Off the Bench is a web app that looks at a photo of an athlete and gives technique ideas. When you finish, a user will be able to:

1. Pick a **sport** and a **skill level**.
2. Upload a photo of a movement (a jump shot, a kick, a dance pose).
3. See the body points the computer found, the joint angles it measured, and **coaching suggestions** that match the sport and level.
4. Optionally ask an **AI model** for extra feedback.

The app already runs. Your job is to write the parts that decide *what the coaching says*: the sports and levels, the suggestions, and the AI prompt.

> **WARNING**
> One photo is one instant. It cannot prove a movement is right or wrong, and angles depend on the camera angle. The app says so everywhere, and you should keep it that way.

Setup

Open a terminal in the `off_the_bench` folder, then run the commands for your computer one at a time.

Mac:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python app.py
```

Windows (PowerShell or Command Prompt):

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
python app.py
```

Open the address it prints (usually `http://127.0.0.1:7860`). To stop the app press `Ctrl + C`. **After every code change, stop and restart the app.** Run the activate line again each time you open a new terminal.

- **Python version:** MediaPipe works with Python 3.9 to 3.12. The app still runs without it; pose detection is just switched off. To turn it on, run `python pose_analysis.py --download` once.
- **AI key (only for Feature 3):** create a Gemini key at aistudio.google.com/apikey and put it in `.env` as `GEMINI_API_KEY=...` with no quotes or spaces. Never paste a key into a Python file or share your `.env`.
- **Windows:** if `python` is not found, try `py`. If PowerShell blocks the activate script, use Command Prompt instead.

The big picture

Information flows through the app like this:

```
you pick a sport + level, upload a photo, click Analyze
                        |
                        v
  app.py  analyze()  runs these steps in order:
                        |
  1. pose_analysis.py   photo -> body landmarks (shoulder, elbow, knee ...)
  2. feedback.py        landmarks -> measurements (elbow angle 95, "bent")
  3. feedback.py        measurements + sport + level -> coaching suggestions
  4. ai_helper.py       (optional) -> prompt -> AI feedback
                        |
                        v
           results shown on the page
```

File	What it does	Yours?
`app.py`	The Gradio page. Its `analyze()` function connects every step above.	Read only
`pose_analysis.py`	Uses MediaPipe to find body points and `calculate_angle` to measure them.	Read only
`sports.py`	The sports, skill levels and level guidance.	**Edit**
`feedback.py`	Labels each measurement and builds the coaching suggestions.	**Edit**
`ai_helper.py`	Builds the prompt and sends it to the AI.	**Edit**

What is a landmark? MediaPipe guesses where 33 body points are in the photo. Each one is a dictionary entry like `landmarks["right_elbow"]`, with its position in pixels and a `visibility` score. You never touch these directly. `feedback.py` turns them into a list of **measurements**, and that list is what you work with:

```
[{"name": "Right elbow angle", "value": 95.0, "label": "bent"}, ...]
```

`value` is `None` when a joint was hidden. **Measurements are facts about the photo. Suggestions are your interpretation of them.** Keep those two ideas separate.

> **NOTE**
> **Note (privacy):** `get_ai_feedback()` only attaches the photo if the user ticks the consent box. Otherwise only text measurements are sent. This is already built in. Do not remove it.

Build the features

You can test everything with pose detection off: the app still shows tips and suggestions. Work in order; the TODOs are numbered in the code.

Feature 1: Sports and skill levels

File: `sports.py` (TODOs 1 and 2)

Goal: The sport dropdown and the level text that the whole app uses come from this file.

Your Task:

- **TODO 1:** Add one more sport to the `SPORTS` dictionary, with a list of skills.
- **TODO 2:** Finish `get_level_guidance`. Beginner is written for you. Write a sentence for intermediate and advanced that tells the coach how much to say and how technical to be.

Hint: `SPORTS` maps a name (the *key*) to a list. The dropdown is built from its keys, so think about what that means for your new sport. For TODO 2, follow the `if` / `elif` / `else` pattern already there.

Test: Restart, then check the dropdown shows your sport. In a terminal: `python -c "import sports; print(sports.get_sport_names()); print(sports.get_level_guidance('advanced'))"` You should see your sport in the list and a real sentence, not `TODO`.

Feature 2: From measurements to coaching

File: `feedback.py` (TODOs 3 and 4)

Goal: Turn raw joint angles into plain-language advice that depends on the athlete's level. `analyze_pose` is provided: it measures the joints that matter for the chosen sport and calls your `classify_angle` for each one.

Your Task:

- **TODO 3:** In `classify_angle`, return one of three labels for an angle: `"nearly straight"`, `"bent"` or `"tightly bent"`. Choose thresholds you can defend (a straight arm or leg is close to 180 degrees).
- **TODO 4:** In `build_suggestions`, loop over the measurements. For each one that has a value, add a suggestion to the list. It should depend on the **label** and on the **level**: gentle and simple for beginners, more specific for advanced athletes. Say "might" and "try", not "wrong" and "correct".

Hint: A measurement is a dictionary, so `m["label"]` and `m["value"]` are the pieces you need. Skip any measurement whose `value` is `None`. An f-string lets you put the joint name and label into your sentence.

Test: Run both functions without a photo: `python -c "from feedback import classify_angle as c; print(c(175), c(110), c(40))"` `python -c "from feedback import build_suggestions as b; print(b('basketball', 'beginner', [{'name': 'Right elbow angle', 'value': 95.0, 'label': 'bent'}]))"` The first line should print three different labels. The second should include a suggestion about the elbow. Then try it for real: with pose detection on, upload a photo of someone shooting (side view, arm visible), click **Analyze**, and read the measurements and suggestions. Change the level and check that the wording changes.

Feature 3: Personalize the AI prompt

File: `ai_helper.py` (TODO 5)

Goal: A prompt is the instruction you send to the AI. A good one says who the athlete is, what data you have, what you want back, and what the AI must **not** claim.

Your Task: In `build_prompt`, improve the returned prompt. The measurement text is built for you. Make the prompt include the sport and level, the level guidance from `get_level_guidance`, a request for 2 to 3 short tips, and an instruction to say when it is unsure and not to claim certainty from a single photo.

Hint: Build the prompt as one long f-string, or join pieces with `+`. `get_level_guidance(level)` is already imported at the top of the file and returns a string you can drop straight in.

Test: Print the prompt and read it as if you were the AI: `python -c "from ai_helper import build_prompt; print(build_prompt('basketball', 'intermediate', [{'name': 'Right elbow angle', 'value': 90.0, 'label': 'bent'}]))"` It should mention basketball, intermediate, the elbow at about 90 degrees, and the level guidance. Then add your key to `.env`, restart, tick **Also ask an AI model**, and click **Analyze**. Try it twice, once with the consent box ticked and once without, and compare the answers. The status line at the top should say `AI feedback is available.`

> **STUCK?**
> If you see `The AI request failed`, read the end of the message: `API key not valid` means a wrong key or a stray space in `.env`. The `.env` file must sit next to `app.py`, and you must restart after editing it.

Optional extensions

- **More sports:** give each sport its own joints in `SPORT_JOINTS` in `feedback.py`, or add a new joint such as the hip.
- **Drills:** add a drill suggestion per skill from `sports.get_skills`, chosen from the measurements.
- **Angle labels:** draw the measured angles on the annotated image.
- **Two photos:** compare a "before" and "after" photo and report which angle changed.
- **Video:** read frames with OpenCV and look at how an angle changes over time. A movement is a sequence, not one pose.